Category C: Encapsulation & Access Control

Java solutions for all five problems. Every class was compiled and run against the sample inputs; the outputs shown are the real results.

Problem 1: The Health Bar

Approach

Health is private and can only change through takeDamage() and heal(). Each method calculates the new value first, then clamps it (Math.max with 0 for damage, Math.min with the maximum for healing). maxHealth is final and set in the constructor. There is a getter but no setter.

Character.java

```
public class Character {
    private final int maxHealth;
    private int health;

    public Character(int maxHealth) {
        if (maxHealth <= 0) {
            throw new IllegalArgumentException("Max health must be positive");
        }
        this.maxHealth = maxHealth;
        this.health = maxHealth;
    }

    public void takeDamage(int amount) {
        if (amount < 0) return;                 // negative damage is ignored
        health = Math.max(0, health - amount);  // calculate, then clamp at 0
    }

    public void heal(int amount) {
        if (amount < 0) return;                       // negative healing is ignored
        health = Math.min(maxHealth, health + amount); // calculate, then clamp at max
    }

    public int getHealth() {      // read-only, no setter
        return health;
    }

    public int getMaxHealth() {
        return maxHealth;
    }
}
```

Output (sample run)

```
c.takeDamage(30) -> health = 70
c.heal(50)       -> health = 100 (capped)
c.takeDamage(150)-> health = 0 (floored)
```

Problem 2: The Playlist

Approach

Songs live in a private array. getSongs() returns a new array built with Arrays.copyOf, containing only the songs added so far, so changing the returned array cannot affect the playlist. getSongCount() returns the internal counter.

Playlist.java

```
import java.util.Arrays;

public class Playlist {
    private final String[] songs;
    private int count;

    public Playlist(int maxSize) {
        if (maxSize <= 0) {
            throw new IllegalArgumentException("Max size must be positive");
        }
        this.songs = new String[maxSize];
        this.count = 0;
    }

    public boolean addSong(String title) {
        if (count == songs.length) return false;   // playlist full
        songs[count++] = title;
        return true;
    }

    // Returns a copy containing only the songs added so far
    public String[] getSongs() {
        return Arrays.copyOf(songs, count);
    }

    public int getSongCount() {
        return count;
    }
}
```

Output (sample run)

```
getSongs() -> [Song A, Song B]
copy[0] = "Hacked"
getSongs() -> [Song A, Song B]   (unchanged)
getSongCount() -> 2
```

Problem 3: The Password Checker

Approach

The password is a private final field with no getter or setter. getStrength() works out the rating from its length: under 6 is Weak, 6 to 9 is Medium, 10 or more is Strong.

PasswordChecker.java

```
public class PasswordChecker {
    private final String password;   // no getter, no setter

    public PasswordChecker(String password) {
        this.password = (password == null) ? "" : password;
    }

    public String getStrength() {
        int len = password.length();
        if (len < 6) return "Weak";
        if (len < 10) return "Medium";
        return "Strong";
    }
}
```

Output (sample run)

```
"abcd"         -> Weak
"abcdefgh"     -> Medium
"abcdefghij"   -> Strong
"abcdefghijkl" -> Strong
```

Problem 4: The Traffic Light

Approach

The current colour is stored as a private index into a fixed array {RED, GREEN, YELLOW}. next() is the only method that changes it, moving forward and wrapping with modulo. There is no setter. The id is final.

TrafficLight.java

```
public class TrafficLight {
    private static final String[] CYCLE = {"RED", "GREEN", "YELLOW"};

    private final String id;
    private int index;   // position in CYCLE; the only way to change it is next()

    public TrafficLight(String id) {
        this.id = id;
        this.index = 0;   // starts on RED
    }

    public String next() {
        index = (index + 1) % CYCLE.length;
        return CYCLE[index];
    }

    public String getColor() {
        return CYCLE[index];
    }

    public String getId() {
        return id;
    }
}
```

Output (sample run)

```
getColor() -> RED
next()     -> GREEN
next()     -> YELLOW
next()     -> RED
next()     -> GREEN
```

Problem 5: The Shopping Cart

Approach

Prices are stored in a private array with a counter for the number of items. getTotal() loops over the array each time it is called, so there is no separate running total to go out of sync. No method returns the array itself. The cart ID is final.

Cart.java

```
public class Cart {
    private final String cartId;
    private final double[] prices;
    private int itemCount;

    public Cart(String cartId, int maxItems) {
        if (maxItems <= 0) {
            throw new IllegalArgumentException("Max items must be positive");
        }
        this.cartId = cartId;
        this.prices = new double[maxItems];
        this.itemCount = 0;
    }

    public boolean addItem(double price) {
        if (price < 0 || itemCount == prices.length) return false;  // invalid or full
        prices[itemCount++] = price;
        return true;
    }

    // Computed on request, not stored separately
    public double getTotal() {
        double total = 0;
        for (int i = 0; i < itemCount; i++) {
            total += prices[i];
        }
        return total;
    }

    public int getItemCount() {
        return itemCount;
    }

    public String getCartId() {
        return cartId;
    }
}
```

Output (sample run)

```
cart.addItem(250); addItem(99); addItem(151)
getTotal()     -> 500
getItemCount() -> 3
```

Test driver (Main.java)

```
import java.util.Arrays;

public class Main {
    public static void main(String[] args) {
        System.out.println("--- Problem 1: Health Bar ---");
        Character c = new Character(100);
        c.takeDamage(30);  System.out.println("takeDamage(30) -> health = " + c.getHealth());
        c.heal(50);        System.out.println("heal(50)       -> health = " + c.getHealth() + " (capped)");
        c.takeDamage(150); System.out.println("takeDamage(150)-> health = " + c.getHealth() + " (floored)");

        System.out.println("--- Problem 2: Playlist ---");
        Playlist p = new Playlist(10);
        p.addSong("Song A"); p.addSong("Song B");
        String[] copy = p.getSongs();
        System.out.println("getSongs() -> " + Arrays.toString(copy));
        copy[0] = "Hacked";
        System.out.println("after modifying copy, getSongs() -> " + Arrays.toString(p.getSongs()));
        System.out.println("getSongCount() -> " + p.getSongCount());

        System.out.println("--- Problem 3: Password Checker ---");
        System.out.println("\"abcd\"         -> " + new PasswordChecker("abcd").getStrength());
        System.out.println("\"abcdefgh\"     -> " + new PasswordChecker("abcdefgh").getStrength());
        System.out.println("\"abcdefghij\"   -> " + new PasswordChecker("abcdefghij").getStrength());
        System.out.println("\"abcdefghijkl\" -> " + new PasswordChecker("abcdefghijkl").getStrength());

        System.out.println("--- Problem 4: Traffic Light ---");
        TrafficLight t = new TrafficLight("TL-9");
        System.out.println("getColor() -> " + t.getColor());
        System.out.println("next()     -> " + t.next());
        System.out.println("next()     -> " + t.next());
        System.out.println("next()     -> " + t.next());
        System.out.println("next()     -> " + t.next());

        System.out.println("--- Problem 5: Shopping Cart ---");
        Cart cart = new Cart("CART-5", 20);
        cart.addItem(250); cart.addItem(99); cart.addItem(151);
        System.out.println("getTotal()     -> " + (int) cart.getTotal());
        System.out.println("getItemCount() -> " + cart.getItemCount());
    }
}
```